UNIVERSIDAD NACIONAL DEL CENTRO DEL PERÚ

FACULTAD DE INGENIERÍA DE SISTEMAS

GUÍA DE LABORATORIO N.º 2

Librerías para Datos y Automatización

I. Propósito de la sesión

Al finalizar el laboratorio, el estudiante será capaz de manipular datos con NumPy usando operaciones vectorizadas, consumir una API REST pública gestionando errores y timeouts, y transformar la respuesta JSON en un DataFrame de Pandas para su análisis y exportación — completando así el primer flujo de datos de principio a fin del curso, sobre el mismo repositorio versionado en la Semana 1.

II. Requisitos previos

- Repositorio tecemer-lab1 de la Semana 1, con su entorno virtual (.venv) y el remoto de GitHub configurados.
- Conexión a internet para consumir la API pública Open-Meteo (no requiere clave de acceso).
- Editor de código Visual Studio Code, con la extensión oficial de Python.
- Haber completado la Guía de Laboratorio N.º 1 (entorno virtual, Git y estructura de proyecto).

III. Estructura de la sesión (4 horas)

Parte	Tema	Nivel	Duración
1	NumPy: fundamentos y computación vectorizada	Básico	60 min
2	Consumo de APIs REST y manejo de JSON	Intermedio	90 min
3	Pandas: de datos crudos a análisis, y consolidación con Git	Avanzado	90 min
Total			**240 min**

IV. Desarrollo paso a paso

PARTE 1 — NUMPY: FUNDAMENTOS Y COMPUTACIÓN VECTORIZADA

Paso 1.1 — Retomar el entorno de la Semana 1 e instalar NumPy

Ubíquese en la carpeta tecemer-lab1 creada la semana pasada e instale NumPy en el mismo entorno virtual:

```
cd tecemer-lab1
.venv/Scripts/pip install numpy
.venv/Scripts/pip freeze > requirements.txt
```

Paso 1.2 — Creación de arrays

Cree un archivo numpy_demo.py con el siguiente contenido:

```
import numpy as np

a = np.array([1, 2, 3, 4, 5])
ceros = np.zeros((2, 3))
rango = np.arange(0, 10, 2)
espaciado = np.linspace(0, 1, 5)

print('Array base:', a)
print('Matriz de ceros:\n', ceros)
print('Rango:', rango)
print('Espaciado:', espaciado)
```

```
.venv/Scripts/python numpy_demo.py
```

Paso 1.3 — Operaciones vectorizadas vs. bucles

Agregue al mismo archivo una comparación cronometrada entre un bucle for y la operación vectorizada equivalente:

```
import numpy as np
import time

n = 1_000_000
lista = list(range(n))
array = np.arange(n)

inicio = time.time()
resultado_lista = [x + 10 for x in lista]
print("Bucle for:", time.time() - inicio, "segundos")

inicio = time.time()
resultado_array = array + 10
print("Vectorizado:", time.time() - inicio, "segundos")
```

Verificación: *Ejecute el script dos o tres veces y compare los tiempos. La versión vectorizada debe ser notablemente más rápida en todas las corridas.*

Paso 1.4 — Mini-reto: estadística descriptiva con NumPy

Simule un arreglo de temperaturas diarias y calcule sus estadísticas básicas:

```
import numpy as np
temperaturas = np.array([12.5, 14.0, 11.8, 13.2, 15.1, 10.9, 12.7])

print('Promedio:', temperaturas.mean())
print('Desviación estándar:', temperaturas.std())
print('Mínima:', temperaturas.min())
print('Máxima:', temperaturas.max())
```

PARTE 2 — CONSUMO DE APIS REST Y MANEJO DE JSON

Paso 2.1 — Repaso de requests con timeout y manejo de errores

Instale requests si aún no está en el entorno (ya se usó en la Semana 1) y repase el patrón de manejo de errores:

```
.venv/Scripts/pip install requests
```

```
import requests

try:
    respuesta = requests.get("https://api.open-meteo.com/v1/forecast", timeout=5)
    respuesta.raise_for_status()
except requests.exceptions.Timeout:
    print('La solicitud excedió el tiempo de espera.')
except requests.exceptions.RequestException as error:
    print(f'Error al consultar la API: {error}')
```

Paso 2.2 — Consumir la API Open-Meteo para Huancayo

Open-Meteo no requiere clave de acceso. Cree un archivo clima.py con el siguiente contenido:

```
import requests
import json

URL = "https://api.open-meteo.com/v1/forecast"
PARAMETROS = {
    "latitude": -12.07,      # Huancayo
    "longitude": -75.21,
    "daily": "temperature_2m_max,temperature_2m_min,precipitation_sum",
    "timezone": "America/Lima",
    "forecast_days": 7,
}

try:
    respuesta = requests.get(URL, params=PARAMETROS, timeout=5)
    respuesta.raise_for_status()
    datos = respuesta.json()
except requests.exceptions.RequestException as error:
    raise SystemExit(f'No se pudo obtener el pronóstico: {error}')

print(json.dumps(datos["daily"], indent=2, ensure_ascii=False))
```

```
.venv/Scripts/python clima.py
```

Salida esperada: *Un objeto JSON con las claves time, temperature_2m_max, temperature_2m_min y precipitation_sum, cada una con 7 valores (uno por día).*

Paso 2.3 — Explorar la estructura anidada del JSON

Agregue al script una exploración de las claves y tipos de datos de la respuesta:

```
print('Claves de primer nivel:', list(datos.keys()))
print('Claves de "daily":', list(datos["daily"].keys()))
print('Tipo de temperature_2m_max:', type(datos["daily"]["temperature_2m_max"]))
```

```
print('Primer valor de tiempo:', datos["daily"]["time"][0])
```

Paso 2.4 — Guardar el JSON crudo y convertir a CSV

Guarde la respuesta cruda (trazabilidad del dato original) y luego constrúyala como CSV con el módulo estándar csv:

```
with open("pronostico_huancayo.json", "w", encoding="utf-8") as archivo:
    json.dump(datos, archivo, ensure_ascii=False, indent=2)
```

```
import csv

diario = datos["daily"]
filas = zip(
    diario["time"],
    diario["temperature_2m_max"],
    diario["temperature_2m_min"],
    diario["precipitation_sum"],
)

with open("pronostico_huancayo.csv", "w", newline="", encoding="utf-8") as archivo:
    escritor = csv.writer(archivo)
    escritor.writerow(["fecha", "temp_max", "temp_min", "precipitacion"])
    escritor.writerows(filas)
```

Verificación del docente: *Checkpoint de Parte 2: deben existir dos archivos nuevos, pronostico_huancayo.json y pronostico_huancayo.csv, ambos con 7 registros de datos.*

PARTE 3 — PANDAS: DE DATOS CRUDOS A ANÁLISIS, Y CONSOLIDACIÓN CON GIT

Paso 3.1 — Instalación de Pandas y carga del CSV

```
.venv/Scripts/pip install pandas
.venv/Scripts/pip freeze > requirements.txt
```

Cree un archivo analisis.py que cargue el CSV generado en la Parte 2:

```
import pandas as pd

df = pd.read_csv("pronostico_huancayo.csv")
df["fecha"] = pd.to_datetime(df["fecha"])
print(df.head())
print(df.info())
```

Paso 3.2 — Exploración y transformación

Agregue columnas derivadas y una clasificación simple de los días:

```
df["amplitud_termica"] = df["temp_max"] - df["temp_min"]
df["dia_lluvioso"] = df["precipitacion"] > 0
df["categoria"] = df["temp_max"].apply(
    lambda t: "cálido" if t >= 20 else ("templado" if t >= 15 else "frío")
)
print(df)
print(df.describe())
```

Paso 3.3 — Agregaciones con groupby

```
resumen = df.groupby("categoria").agg(
    dias=("categoria", "count"),
    temp_max_promedio=("temp_max", "mean"),
    precipitacion_total=("precipitacion", "sum"),
)
print(resumen)
```

Paso 3.4 — Exportación de resultados y actualización del README

```
df.to_csv("pronostico_huancayo_procesado.csv", index=False)
resumen.to_csv("resumen_por_categoria.csv")
```

Agregue al README.md una sección nueva describiendo el flujo de datos de esta semana (fuente, transformación, salida). Luego confirme los cambios:

```
git add numpy_demo.py clima.py analisis.py *.csv *.json README.md requirements.txt
git commit -m "feat: agrega pipeline de datos con NumPy, requests y Pandas (pronostico Huancayo)"
```

Paso 3.5 — Consolidación final y entrega

```
git push origin main
```

Copie el enlace de su repositorio (ya con el historial acumulado de las Semanas 1 y 2) y súbalo al aula virtual como evidencia de entrega.

V. Entregable de la sesión

El mismo repositorio de GitHub de la Semana 1, actualizado con: numpy_demo.py, clima.py y analisis.py; los archivos de datos pronostico_huancayo.json, pronostico_huancayo.csv, pronostico_huancayo_procesado.csv y resumen_por_categoria.csv; el README.md actualizado con la nueva sección de flujo de datos; y un historial de commits que documenta el avance en formato Conventional Commits.